```
import sys
from fractions import Fraction
from itertools import permutations, product
from collections import Counter, defaultdict
from cmath import *     # x + y * 1j

def readint(): return int(sys.stdin.readline())
def readstr(): return sys.stdin.readline().strip()
def readints(): return list(map(int, readstr().split()))

print(f"{answer=}", file=sys.stderr)  # debug
```

## check_hash.py

```
import ast, hashlib, sys    # les commentaires sont ignorés pour le code de hashage
with open(sys.argv[1], encoding="utf-8") as f:
    code = f.read()
normalized = ast.unparse(ast.parse(code))  + "\n"
hashcode = hashlib.md5(normalized.encode("utf-8")).hexdigest()
print(hashcode[:6])
```

## Anagrams

3115b8

```
from collections import defaultdict

def anagrams2(S):                   # S is a set of strings
    """group a set of words into anagrams

    :param S: set of strings
    :returns: list of lists of strings

    :complexity:
        :math:`O(n k log k)` in average, for n words of length at most k.
        :math:`O(n^2 k log k)` in worst case due to the usage of a dictionary.
    """
    d = defaultdict(list)                   # maps s to list of words with
signature s
    for w in S:                             # group words according to the
signature
        d[''.join(sorted(w))].append(w)     # calculate the signature
    # -- extract anagrams, ignoring anagram groups of size 1
    return [v for v in d.values() if len(v) > 1]
```

## Arithm Expr Eval

fa6cab

```
def arithm_expr_eval(cell, expr):
    """Evaluates a given expression

    :param expr: expression
    :param cell: dictionary variable name -> expression

    :returns: numerical value of expression

    :complexity: linear
    """
    if isinstance(expr, tuple):
        (left, operand, right) = expr
        lval = arithm_expr_eval(cell, left)
        rval = arithm_expr_eval(cell, right)
        if operand == '+':
            return lval + rval
        if operand == '-':
            return lval - rval
        if operand == '*':
            return lval * rval
        if operand == '/':
            return lval // rval
    elif isinstance(expr, int):
        return expr
    else:
        cell[expr] = arithm_expr_eval(cell, cell[expr])
        return cell[expr]
```

5279a4

```
PRIORITY = {';': 0, '(': 1, ')': 2, '-': 3, '+': 3, '*': 4, '/': 4}


def arithm_expr_parse(line_tokens):
    """Constructs an arithmetic expression tree

    :param line_tokens: list of token strings containing the expression
    :returns: expression tree

    :complexity: linear
    """
    vals = []
    ops = []
    for tok in line_tokens + [';']:
        if tok in PRIORITY:  # tok is an operator
            while (tok != '(' and ops and
                   PRIORITY[ops[-1]] >= PRIORITY[tok]):
                right = vals.pop()
                left = vals.pop()
                vals.append((left, ops.pop(), right))
            if tok == ')':
                ops.pop()    # this is the corresponding '('
            else:
                ops.append(tok)
        elif tok.isdigit():  # tok is an integer
            vals.append(int(tok))
        else:                # tok is an identifier
            vals.append(tok)
    return vals.pop()
```

## Bellman Ford

48fabd

```
def bellman_ford(graph, weight, source=0):
    """ Single source shortest paths by Bellman-Ford
```

```
    :param graph: directed graph in listlist or listdict format
    :param weight: can be negative.
                   in matrix format or same listdict graph
    :returns: distance table, precedence table, bool,
                None means no predecessor
    :explanation: bool is True if a negative circuit is
                  reachable from the source, circuits
                  can have length 2.
    :complexity: `O(|V|*|E|)`
    """
    n = len(graph)
    dist = [float('inf')] * n
    prec = [None] * n
    dist[source] = 0
    for nb_iterations in range(n):
        changed = False
        for node in range(n):
            for neighbor in graph[node]:
                alt = dist[node] + weight[node][neighbor]
                if alt < dist[neighbor]:
                    dist[neighbor] = alt
                    prec[neighbor] = node
                    changed = True
        if not changed:                    # fixed point
            return dist, prec, False
    return dist, prec, True
```

```
def bellman_ford_edges(edges, n, s=0):                                  e1da31
    """ Single source shortest paths by Bellman-Ford
    does not detect negative cycles

    :param edge: list of arcs (u,v) with their weight
    :param n: vertices are numbered from 0 to n - 1
    :returns: list d where d[u][0] is distance from s
              and d[u][1] is the predecessor on shortest path,
              -1 means no predecessor
    :complexity: `O(|V|*|E|)`
    """
    d = [(float('inf'), -1)] * n
    d[s] = (0, -1)
    for _ in range(n):
        for u, v, w in edges:
            d[v] = min(d[v], (d[u][0] + w, u))
    return d
```

## Bezout

```
def bezout(a, b):                                                       a984a7
    """Bézout coefficients for a and b

    :param a,b: non-negative integers
    :complexity: O(log a + log b)
    """
    if b == 0:
        return (1, 0)
    u, v = bezout(b, a % b)
    return (v, u - (a // b) * v)
```

```
pow(a, b, q)        # fast_exponentiation                               977741
pow(a, -1, q)       # inv(a, q), Python >= 3.8
math.gcd(a, b)      # pgcd
math.comb(n, k)     # binom
```

## Bfs

```
from collections import deque                                           fecba9


def bfs(graph, start=0):
    """Shortest path in unweighted graph by BFS

       :param graph: directed graph in listlist or listdict format
       :param int start: source vertex
       :returns: distance table, precedence table
       :complexity: `O(|V|+|E|)`
       """
    to_visit = deque()
    dist = [float('inf')] * len(graph)
    prec = [None] * len(graph)
    dist[start] = 0
    to_visit.appendleft(start)
    while to_visit:              # an empty queue is considered False
        node = to_visit.pop()
        for neighbor in graph[node]:
            if dist[neighbor] == float('inf'):
                dist[neighbor] = dist[node] + 1
                prec[neighbor] = node
                to_visit.appendleft(neighbor)
    return dist, prec
```

## Biconnected Components

```
# to ease readiness, variables do not have dfs_ prefix                  071332
def cut_nodes_edges(graph):
    """Bi-connected components

    :param graph: undirected graph. in listlist format.
                  Cannot be in listdict format.
    :returns: a tuple with the list of cut-nodes and the list of cut-edges
    :complexity: `O(|V|+|E|)`
    """
    n = len(graph)
    time = 0
    num = [None] * n
    low = [n] * n
    parent = [None] * n        # parent[v] = None if root else parent of v
    critical_children = [0] * n  # cc[u] = #{children v | low[v] >= num[u]}
    times_seen = [-1] * n
    for start in range(n):
```

```
        if times_seen[start] == -1:               # init DFS path
            times_seen[start] = 0
            to_visit = [start]
            while to_visit:
                node = to_visit[-1]
                if times_seen[node] == 0:         # start processing
                    num[node] = time
                    time += 1
                    low[node] = float('inf')
                children = graph[node]
                if times_seen[node] == len(children):  # end processing
                    to_visit.pop()
                    up = parent[node]             # propagate low to parent
                    if up is not None:
                        low[up] = min(low[up], low[node])
                        if low[node] >= num[up]:
                            critical_children[up] += 1
                else:
                    child = children[times_seen[node]]   # next arrow
                    times_seen[node] += 1
                    if times_seen[child] == -1:   # not visited yet
                        parent[child] = node      # link arrow
                        times_seen[child] = 0
                        to_visit.append(child)    # (below) back arrow
                    elif num[child] < num[node] and parent[node] != child:
                        low[node] = min(low[node], num[child])
    cut_edges = []
    cut_nodes = []                                # extract solution
    for node in range(n):
        if parent[node] is None:                  # characteristics
            if critical_children[node] >= 2:
                cut_nodes.append(node)
        else:                                     # internal nodes
            if critical_children[node] >= 1:
                cut_nodes.append(node)
            if low[node] >= num[node]:
                cut_edges.append((parent[node], node))
    return cut_nodes, cut_edges
```

## Binary Search

```
def discrete_binary_search(tab, lo, hi):                                6c8d91
    """Binary search in a table

    :param tab: boolean monotone table with tab[hi] = True
    :param int lo:
    :param int hi: with hi >= lo
    :returns: first index i in [lo,hi] such that tab[i]
    :complexity: `O(log(hi-lo))`
    """
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if tab[mid]:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

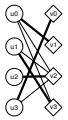
```
def continuous_binary_search(f, lo, hi, gap=1e-4):                      4b5661
    """Binary search for a function

    :param f: boolean monotone function with f(hi) = True
    :param int lo:
    :param int hi: with hi >= lo
    :param float gap:
    :returns: first value x in [lo,hi] such that f(x),
            x is computed up to some precision
    :complexity: `O(log((hi-lo)/gap))`
    """
    while hi - lo > gap:
        mid = (lo + hi) / 2.0
        if f(mid):
            hi = mid
        else:
            lo = mid
    return lo
```

```
def optimized_binary_search(tab, logsize):                              39d263
    """Binary search in a table using bit operations

    :param tab: boolean monotone table
       of size :math:`2^\\textrm{logsize}`
       with tab[hi] = True
    :param int logsize:
    :returns: first i such that tab[i]
    :complexity: O(logsize)
    """
    hi = (1 << logsize) - 1
    intervalsize = (1 << logsize) >> 1
    while intervalsize > 0:
        if tab[hi ^ intervalsize]:
            hi ^= intervalsize
        intervalsize >>= 1
    return hi
```

## Bipartite Matching



```
def augment(u, bigraph, visit, timestamp, match):                       07a5f4
    """ find augmenting path starting from u, by recursive DFS """
    for v in bigraph[u]:
        if visit[v] < timestamp:
            visit[v] = timestamp
            if match[v] is None or augment(match[v], bigraph,
```

```
                                        visit, timestamp, match):
                match[v] = u       # found an augmenting path
                return True
    return False


def max_bipartite_matching(bigraph):
    """Bipartie maximum matching

    :param bigraph: adjacency list, index = vertex in U,
                                    value = neighbor list in V
    :comment: U and V can have different cardinalities
    :returns: matching list, match[v] == u iff (u, v) in matching
    :complexity: `O(|V|*|E|)`
    """
    nU = len(bigraph)   # nU = cardinality of U, nV = card. of V
    nV = max(max(adjlist, default=-1) for adjlist in bigraph) + 1
    match = [None] * nV
    visit = [-1] * nV   # timestamp of last visit
    for u in range(nU):
        augment(u, bigraph, visit, u, match)
    return match
```

## Convex Hull



```
def left_turn(a, b, c):                                                e40e99
    """function left-turn"""
    return ((a[0] - c[0]) * (b[1] - c[1]) -
            (a[1] - c[1]) * (b[0] - c[0]) > 0)
```

```
def andrew(S):                                                         b224fa
    """Convex hull by Andrew

    :param S: list of points as coordinate pairs
    :requires: S has at least 2 points
    :returns: list of points of the convex hull
    :complexity: `O(n log n)`
    """
    S.sort()
    top = []
    bot = []
    for p in S:
        while len(top) >= 2 and not left_turn(p, top[-1], top[-2]):
            top.pop()
        top.append(p)
        while len(bot) >= 2 and not left_turn(bot[-2], bot[-1], p):
            bot.pop()
        bot.append(p)
    return bot[:-1] + top[:0:-1]
```

## Dfs

```
def dfs_recursive(graph: Graph, node: int, seen: List[bool]) -> None:  5ce1f8
    """DFS, detect connected component, recursive implementation

    :param graph: directed graph in listlist or listdict format
    :param int node: to start graph exploration
    :param boolean-table seen: will be set true for the connected component
          containing node.
    :complexity: `O(|V|+|E|)`
    """
    seen[node] = True
    for neighbor in graph[node]:
        if not seen[neighbor]:
            dfs_recursive(graph, neighbor, seen)
```

```
def dfs_iterative(graph: Graph, start: int, seen: List[bool]) -> None: 9ba911
    """DFS, detect connected component, iterative implementation

    :param graph: directed graph in listlist or listdict format
    :param int node: to start graph exploration
    :param boolean-table seen: will be set true for the connected component
          containing node.
    :complexity: `O(|V|+|E|)`
    """
    to_visit = [start]
    while to_visit:
        node = to_visit.pop()   # exploring in top-down order of stack
        if not seen[node]:      # hence the use of reversed
            to_visit.extend(reversed(graph[node]))
            seen[node] = True   # vertex can be multiple times on stack
```

```
def dfs_tree(graph: Graph, start: int=0) -> List[Optional[int]]:       3aeadf
    """DFS, build DFS tree in unweighted graph

       :param graph: directed graph in listlist or listdict format
       :param int start: source vertex
       :returns: precedence table
       :complexity: `O(|V|+|E|)`
       """
    to_visit = [start]
    prec: List[Optional[int]] = [None] * len(graph)
    while to_visit:             # an empty queue equals False
        node = to_visit.pop()
        for neighbor in graph[node]:
            if prec[neighbor] is None:
                prec[neighbor] = node
                to_visit.append(neighbor)
    return prec
```

```
def dfs_grid(grid, i, j, mark='X', free: str='.') -> None:             d2b771
    """DFS on a grid, mark connected component, iterative version

    :param grid: matrix, 4-neighborhood
```

```
    :param i,j: cell in this matrix, start of DFS exploration
    :param free: symbol for walkable cells
    :param mark: symbol to overwrite visited vertices
    :complexity: linear
    """
    height = len(grid)
    width = len(grid[0])
    to_visit = [(i, j)]
    grid[i][j] = mark
    while to_visit:
        i1, j1 = to_visit.pop()
        for i2, j2 in [(i1 + 1, j1), (i1, j1 + 1),
                       (i1 - 1, j1), (i1, j1 - 1)]:
            if (0 <= i2 < height and 0 <= j2 < width and
                    grid[i2][j2] == free):
                grid[i2][j2] = mark  # mark path
                to_visit.append((i2, j2))
```

## Dijkstra

```
def dijkstra(g, w, s, t=-1):                    028907
    d = [float('inf')] * len(g); d[s] = 0   # distance from s
    p = [None] * len(g)                     # predecessor on shortest path
    h = [(0, s)]                            # heap
    while h:
        du, u = heappop(h)
        if du > d[u]: continue              # not shortest path to u
        if u == t: break                    # target reached, done
        for v in g[u]:
            if du + w[u][v] < d[v]:         # improves distance
                d[v] = du + w[u][v]
                p[v] = u
                heappush(h, (d[v], v))
    return d, p
```

## Dilworth

```
def dilworth(graph):                            9d3de9
    """Decompose a DAG into a minimum number of chains by Dilworth

    :param graph: directed graph in listlist or listdict format
    :assumes: graph is acyclic
    :returns: table giving for each vertex the number of its chains
    :complexity: same as matching
    """
    n = len(graph)
    match = max_bipartite_matching(graph)  # maximum matching
    part = [None] * n                      # partition into chains
    nb_chains = 0
    for v in range(n - 1, -1, -1):         # in inverse topological order
        if part[v] is None:                # start of chain
            u = v
            while u is not None:           # follow the chain
                part[u] = nb_chains        # mark
                u = match[u]
            nb_chains += 1
    return part
```

## Dinic

```
def dinic(graph, capacity, source, target):     959d1b
    """Maximum flow by Dinic

    :param graph: directed graph in listlist or listdict format
    :param capacity: in matrix format or same listdict graph
    :param int source: vertex
    :param int target: vertex
    :returns: skew symmetric flow matrix, flow value
    :complexity: :math:`O(|V|^2 |E|)`
    """
    assert source != target
    add_reverse_arcs(graph, capacity)
    Q = deque()
    total = 0
    n = len(graph)
    flow = [[0] * n for u in range(n)]   # flow initially empty
    while True:                          # repeat while we can increase
        Q.appendleft(source)
        level = [None] * n               # build levels, None = inaccessible
        level[source] = 0                # by BFS
        while Q:
            u = Q.pop()
            for v in graph[u]:
                if level[v] is None and capacity[u][v] > flow[u][v]:
                    level[v] = level[u] + 1
                    Q.appendleft(v)

        if level[target] is None:   # stop if sink is not reachable
            return flow, total
        up_bound = sum(capacity[source][v] for v in graph[source]) - total
        total += _dinic_step(graph, capacity, level, flow, source, target,
                             up_bound)


def _dinic_step(graph, capacity, level, flow, u, target, limit):
    """ tenter de pousser le plus de flot de u à target, sans dépasser limit
    """
    if limit <= 0:
        return 0
    if u == target:
        return limit
    val = 0
    for v in graph[u]:
        residual = capacity[u][v] - flow[u][v]
        if level[v] == level[u] + 1 and residual > 0:
            z = min(limit, residual)
            aug = _dinic_step(graph, capacity, level, flow, v, target, z)
            flow[u][v] += aug
            flow[v][u] -= aug
            val += aug
            limit -= aug
```

```
    if val == 0:
        level[u] = None         # remove unreachable node
    return val
```

## Edmonds Karp

```
def _augment(graph, capacity, flow, source, target):                    eaed21
    """find a shortest augmenting path
    """
    n = len(graph)
    A = [0] * n                 # A[v] = min residual cap. on path source->v
    augm_path = [None] * n      # None = node was not visited yet
    Q = deque()                 # BFS
    Q.append(source)
    augm_path[source] = source
    A[source] = float('inf')
    while Q:
        u = Q.popleft()
        for v in graph[u]:
            cuv = capacity[u][v]
            residual = cuv - flow[u][v]
            if residual > 0 and augm_path[v] is None:
                augm_path[v] = u    # store predecessor
                A[v] = min(A[u], residual)
                if v == target:
                    break
                Q.append(v)
    return (augm_path, A[target])   # augmenting path, min residual cap.


def edmonds_karp(graph, capacity, source, target):
    """Maximum flow by Edmonds-Karp

    :param graph: directed graph in listlist or listdict format
    :param capacity: in matrix format or same listdict graph
    :param int source: vertex
    :param int target: vertex
    :returns: flow matrix, flow value
    :complexity: :math:`O(|V|*|E|^2)`
    """
    add_reverse_arcs(graph, capacity)
    V = range(len(graph))
    flow = [[0 for v in V] for u in V]
    while True:
        augm_path, delta = _augment(graph, capacity, flow, source, target)
        if delta == 0:
            break
        v = target                      # go back to source
        while v != source:
            u = augm_path[v]            # augment flow
            flow[u][v] += delta
            flow[v][u] -= delta
            v = u
    return (flow, sum(flow[source]))  # flow network, amount of flow
```

## Fast Exponentiation

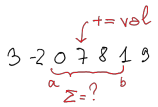
```
def matrix_mul(A, B, q):                                                7f92fe
    """ multiples matrix A with B modulo q

    :param A, B: n by n matrix
    :complexity: O(n^3)
    """
    dim = range(len(A))
    AB = [[0 for j in dim] for i in dim]
    for i in dim:
        for j in dim:
            for k in dim:
                AB[i][j] = (AB[i][j] + A[i][k] * B[k][j]) % q
    return AB


def matrix_fast_exponentiation(A, b, q):
    """Compute (A pow b) % q

    :param A: n by n matrix
    :param int b, q: positive integer
    :complexity: O(log b)
    """
    dim = range(len(A))
    result = [[int(i == j) for j in dim] for i in dim]
    while b:
        if b % 2 == 1:
            result = matrix_mul(result, A, q)
        A = matrix_mul(A, A, q)
        b >>= 1
    return result
```

## Fenwick




```
class Fenwick:                                                          04cc20
    """maintains a tree to allow quick updates and queries
    """
    def __init__(self, t):
        """stores a table t and allows updates and queries
        of prefix sums in logarithmic time.

        :param array t: with numerical values
        """
        self.s = [0] * (len(t) + 1)  # create internal storage
        for a, v in enumerate(t):
            self.add(a, v)           # initialize

    def prefixSum(self, a):
        """
        :param int a: index in t, negative a will return 0
```

```
        :returns: t[0] + ... + t[a]
        """
        i = a + 1                   # internal index starts at 1
        total = 0
        while i > 0:                # loops over neighbors
            total += self.s[i]      # cumulative sum
            i -= (i & -i)           # left neighbor
        return total

    def intervalSum(self, a, b):
        """
        :param int a b: with 0 <= a <= b
        :returns: t[a] + ... + t[b]
        """
        return self.prefixSum(b) - self.prefixSum(a-1)

    def add(self, a, val):
        """
        :param int a: index in t
        :modifies: adds val to t[a]
        """
        i = a + 1                   # internal index starts at 1
        while i < len(self.s):      # loops over parents
            self.s[i] += val        # update node
            i += (i & -i)           # parent

    # variante:
    def intervalAdd(self, a, b, val):
        """Variant, adds val to t[a], to t[a + 1] ... and to t[b]

        :param int a b: with 0 <= a <= b < len(t)
        """
        self.add(a,     +val)
        self.add(b + 1, -val)

    def get(self, a):
        """Variant, reads t[a]

        :param int i: negative a will return 0
        """
        return self.prefixSum(a)
```

## Fft

```
import math     # for pi
import cmath    # for exp


def pad(x):
    """ pad array x with zeros to make its length a power of two
    :complexity: linear
    """
    n = 1
    while n < len(x):
        n <<= 1
    x += [0] * (n - len(x))
```

24d447

```
def fft(x):
    """ Fast Fourier Transformation
    :input x: list of coefficients. Length n has to be a power of 2.
    :returns: list of sample values.
    :complexity: O(n log n).
    """
    n2 = len(x) // 2
    if n2 == 0:
        return x
    assert(2 * n2 == len(x))        # need to split evenly
    even = fft(x[0::2])
    odd  = fft(x[1::2])
    T = [cmath.exp(-1j * math.pi * k / n2) * odd[k] for k in range(n2)]
    return [even[k] + T[k] for k in range(n2)] + \
           [even[k] - T[k] for k in range(n2)]


def inv_fft(y):
    """ Inverse Fast Fourier Transformation
    :input y: list of sample values. Length n has to be a power of 2.
    :returns: list of coefficients.
    :complexity: O(n log n).
    """
    n = len(y)
    p = fft([yi.conjugate() for yi in y])
    return [pi.conjugate() / n for pi in p]


def mul_poly_fft(p, q):
    """Multiply two polynomials in integer coefficient representation
    :complexity: O(n log n)
    """
    n = (len(p) + len(q))   # make them of same and enough large size
    p += [0] * (n - len(p))
    q += [0] * (n - len(q))
    pad(p)
    y = fft(p)
    pad(q)
    z = fft(q)
    n = len(y)  # the padding might have increased the size n
    r = [y[i] * z[i] for i in range(n)]
    R = inv_fft(r)
    return [int(round(ri.real)) for ri in R]
```

## Floyd Warshall

```
def floyd_warshall(weight):
    """All pairs shortest paths by Floyd-Warshall

    :param weight: edge weight matrix
    :modifies: weight matrix to contain distances in graph
    :returns: True if there are negative cycles
    :complexity: :math:`O(|V|^3)`
    """
```

64b20a

```
    V = range(len(weight))
    for k in V:
        for u in V:
            for v in V:
                weight[u][v] = min(weight[u][v],
                                   weight[u][k] + weight[k][v])
    for v in V:
        if weight[v][v] < 0:      # negative cycle found
            return True
    return False
```

## Ford Fulkerson

```
def _augment(graph, capacity, flow, val, u, target, visit, timestamp):
    """Find an augmenting path from u to target with value at most val"""
    visit[u] = timestamp
    if u == target:
        return val
    for v in graph[u]:
        cuv = capacity[u][v]
        if visit[v] < timestamp and cuv > flow[u][v]:  # reachable arc
            res = min(val, cuv - flow[u][v])
            delta = _augment(graph, capacity, flow, res, v, target, visit,
timestamp)
            if delta > 0:
                flow[u][v] += delta            # augment flow
                flow[v][u] -= delta
                return delta
    return 0


def ford_fulkerson(graph, capacity, s, t):
    """Maximum flow by Ford-Fulkerson

    :param graph: directed graph in listlist or listdict format
    :param capacity: in matrix format or same listdict graph
    :param int s: source vertex
    :param int t: target vertex

    :returns: flow matrix, flow value
    :complexity: `O(|V|*|E|*max_capacity)`
    """
    add_reverse_arcs(graph, capacity)
    n = len(graph)
    flow = [[0] * n for _ in range(n)]
    INF = float('inf')
    visit = [-1] * n
    timestamp = 0
    while _augment(graph, capacity, flow, INF, s, t, visit, timestamp) > 0:
        timestamp += 1
    return (flow, sum(flow[s]))      # flow network, amount of flow
```

22f315

## Gale Shapley

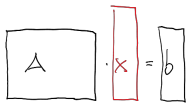
```
from collections import deque


def gale_shapley(men, women):
    """Stable matching by Gale-Shapley

    :param men: table of size n, men[i] is preference list of women for men i
    :param women: similar
    :returns: matching table, from women to men
    :complexity: :math:`O(n^2)`
    """
    n = len(men)
    assert n == len(women)
    current_suitor = [0] * n            # nb of matchings so far
    spouse = [None] * n                 # current matching
    rank = [[0] * n for j in range(n)]  # build rank
    for j in range(n):
        for r in range(n):
            rank[j][women[j][r]] = r
    singles = list(range(n))            # initially all men are single
    while singles:
        i = singles.pop()
        j = men[i][current_suitor[i]]   # propose matching (i,j)
        current_suitor[i] += 1
        if spouse[j] is None:
            spouse[j] = i
        elif rank[j][spouse[j]] < rank[j][i]:
            singles.append(i)
        else:
            singles.append(spouse[j])  # sorry for spouse[j]
            spouse[j] = i
    return spouse
```

d82188

## Gauss Jordan




```
def is_zero(x):                     # tolerance
    """error tolerant zero test
    """
    return -1e-6 < x and x < 1e-6
    # replace with x == 0 si we are handling Fraction elements


GJ_ZERO_SOLUTIONS = 0
GJ_SINGLE_SOLUTION = 1
GJ_SEVERAL_SOLUTIONS = 2


def gauss_jordan(A, x, b):
    """Linear equation system Ax=b by Gauss-Jordan
```

2380d0

```
    :param A: m by n matrix
    :param x: table of size n
    :param b: table of size m
    :modifies: x will contain solution if any
    :returns int:
          0 if no solution,
          1 if solution unique,
          2 otherwise
    :complexity: :math:`O(n^2m)`
    """
    n = len(x)
    m = len(b)
    assert len(A) == m and len(A[0]) == n
    S = []                          # put linear system in a single matrix S
    for i in range(m):
        S.append(A[i][:] + [b[i]])
    S.append(list(range(n)))        # indices in x
    k = diagonalize(S, n, m)
    if k < m:
        for i in range(k, m):
            if not is_zero(S[i][n]):
                return GJ_ZERO_SOLUTIONS
    for j in range(k):
        x[S[m][j]] = S[j][n]
    if k < n:
        for j in range(k, n):
            x[S[m][j]] = 0
        return GJ_SEVERAL_SOLUTIONS
    return GJ_SINGLE_SOLUTION


def diagonalize(S, n, m):
    """diagonalize """
    for k in range(min(n, m)):
        val, i, j = max((abs(S[i][j]), i, j)
                        for i in range(k, m) for j in range(k, n))
        if is_zero(val):
            return k
        S[i], S[k] = S[k], S[i]     # swap lines k, i
        for r in range(m + 1):      # swap columns k, j
            S[r][j], S[r][k] = S[r][k], S[r][j]
        pivot = float(S[k][k])      # without float if Fraction elements
        for j in range(k, n + 1):
            S[k][j] /= pivot        # divide line k by pivot
        for i in range(m):          # remove line k scaled by line i
            if i != k:
                fact = S[i][k]
                for j in range(k, n + 1):
                    S[i][j] -= fact * S[k][j]
    return min(n, m)
```

## Graph01

```
def dist01(graph, weight, source=0, target=None):                    00a690
    """Shortest path in a 0,1 weighted graph
    :param graph: directed graph in listlist or listdict format
    :param weight: matrix or adjacency dictionary
    :param int source: vertex
    :param target: exploration stops once distance to target is found
    :returns: distance table, predecessor table
    :complexity: `O(|V|+|E|)`
    """
    n = len(graph)
    dist = [float('inf')] * n
    prec = [None] * n
    black = [False] * n
    dist[source] = 0
    gray = deque([source])
    while gray:
        node = gray.pop()
        if black[node]:
            continue
        black[node] = True
        if node == target:
            break
        for neighbor in graph[node]:
            ell = dist[node] + weight[node][neighbor]
            if black[neighbor] or dist[neighbor] <= ell:
                continue
            dist[neighbor] = ell
            prec[neighbor] = node
            if weight[node][neighbor] == 0:
                gray.append(neighbor)
            else:
                gray.appendleft(neighbor)
    return dist, prec
```

## Hamiltonian Cycle

```
def hamiltonian_cycle(weight):                                       65b5f6
    """Hamiltonian Cycle

    :param weight: matrix of edge weights of a complete graph
    :returns: minimum weight of a Hamiltonian cycle
    :complexity: O(n^2 2^n)
    """
    n = len(weight)
    # A[S][v] = minimum weight path from vertex n-1,
    #           then visiting all vertices in S exactly once,
    #           and finishing in v (which is not in S)
    A = [[float('inf')] * n for _ in range(1 << (n - 1))]
    for v in range(n):                  # base case
        A[0][v] = weight[n - 1][v]
    for S in range(1, 1 << (n - 1)):
        for v in range(n):
            if not ((1 << v) & S):      # v not in S
                for u in range(n - 1):
                    U = 1 << u
                    if U & S:           # u in S
                        alt = A[S ^ U][u] + weight[u][v]
```

```
                        if alt < A[S][v]:
                            A[S][v] = alt
    S = (1 << (n - 1)) - 1              # {0, 1, ..., n - 2}
    return A[S][n - 1]
```

## Hierholzer



```
def hierholzer_directed(graph):                                    120423
    """Eulerian tour of a directed graph.

    :assumes: graph is Eulerian
    :returns: Eulerian cycle as a vertex list
    :complexity: O(|V| + |E|)
    """
    pos = [0] * len(graph)
    stack = [0]
    tour = []
    while stack:
        u = stack[-1]
        if pos[u] < len(graph[u]):
            v = graph[u][pos[u]]
            pos[u] += 1
            stack.append(v)
        else:
            tour.append(stack.pop())
    return tour[::-1]
```

```
def hierholzer_undirected(graph):                                  ea5a51
    """Eulerian tour of an undirected graph.

    :assumes: graph is Eulerian
    :returns: Eulerian cycle as a vertex list
    :complexity: O(|V| + |E|)
    """
    pos = [0] * len(graph)
    seen = [set() for _ in graph]
    stack = [0]
    tour = []
    while stack:
        u = stack[-1]
        if pos[u] == len(graph[u]):
            tour.append(stack.pop())
        else:
            v = graph[u][pos[u]]
            pos[u] += 1
            if v not in seen[u]:
                seen[v].add(u)
                stack.append(v)
    return tour[::-1]
```

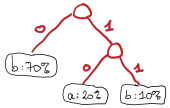
## Horn Sat

```
def horn_sat(formula):                                             a4bd23
    """ Solving a HORN Sat formula

    :param formula: list of couple(posvar, negvars).
                    negvars is a list of the negative variables (can be empty)
                    posvar is the positive variable (can be None)
                    Variables can be any hashable objects: integers, strings...
    :returns: None if formula is not satisfiable, else a minimal set of vars
              that have to be set to true in order to satisfy the formula.
    :complexity: linear
    """
    # --- construct data structures
    CLAUSES = range(len(formula))
    score = [0 for c in CLAUSES]  # number of neg vars not yet in solution
    # the unique positive variable of a clause (if any)
    posvar_in_clause = [None for c in CLAUSES]
    # all clauses where a variable appears negatively
    clauses_with_negvar = defaultdict(set)
    for c in CLAUSES:
        posvar, negvars = formula[c]
        score[c] = len(set(negvars))  # do not count twice negative variables
        posvar_in_clause[c] = posvar
        for v in negvars:
            clauses_with_negvar[v].add(c)
    pool = [set() for s in range(max(score) + 1)]   # create the pool
    for c in CLAUSES:
        pool[score[c]].add(c)           # pool[s] = set of clauses with score s

    # --- solve Horn SAT formula
    solution = set()                    # contains all variables set to True
    while pool[0]:
        curr = pool[0].pop()                # arbitrary zero score clause
        v = posvar_in_clause[curr]
        if v is None:                       # formula is not satisfiable
            return None
        if v in solution or curr in clauses_with_negvar[v]:
            continue                        # clause is already satisfied
        solution.add(v)
        for c in clauses_with_negvar[v]:    # update score
            pool[score[c]].remove(c)
            score[c] -= 1
            pool[score[c]].add(c)           # change c to lower score in pool
    return solution
```

## Huffman




```
from heapq import heappush, heappop                                a4432f

def huffman(freq):
```

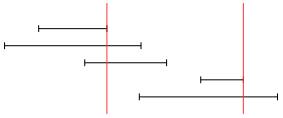
```
    """Huffman code

    :param freq: dictionary with frequencies for each item
    :returns: dictionary with binary code string for each item
    :complexity: O(n log n), where n = len(freq)
    """
    forest = []     # is a heap with (frequence, tree_index) pairs
    trees = []      # list of trees, tree is a letter or a list of two trees
    for item in freq:
        heappush(forest, (freq[item], len(trees)))
        trees.append(item)  # this item will be at index len(trees)
    while len(forest) > 1:
        (f_l, left) = heappop(forest)        # merge two trees
        (f_r, right) = heappop(forest)
        heappush(forest, (f_l + f_r, len(trees)))
        trees.append([trees[left], trees[right]])
    code = {}                                # build code from unique tree
    extract(code, trees[forest[0][1]], [])
    return code


def extract(code, tree, prefix):
    """Extract Huffman code from a Huffman tree

    :param code: a dictionary that will contain the constructed code.
                 should initially be empty.
    :param tree: a node of the tree.
                 Leafs are of the form (frequency, symbol).
                 Inner nodes are of the form [left_sub_tree, right_sub_tree].
    :param prefix: a list with the 01 characters encoding the path from
                   the root to the node `tree`
    :complexity: O(n), where n = number of nodes in tree
    """
    if isinstance(tree, list):        # inner node
        left, right = tree
        prefix.append('0')
        extract(code, left, prefix)   # build code recursively
        prefix.pop()
        prefix.append('1')
        extract(code, right, prefix)  # ... on both subtrees
        prefix.pop()
    else:
        code[tree] = ''.join(prefix)  # extract codeword from prefix
```

## Interval Cover



```
def interval_cover(I):                                                  cdddf8
    """Minimum interval cover

    :param I: list of closed intervals
    :returns: minimum list of points covering all intervals
    :complexity: O(n log n)
    """
    S = []
    # sort by right endpoints
    for start, end in sorted(I, key=lambda v: v[1]):
        if not S or S[-1] < start:
            S.append(end)
    return S
```

## Karatsuba

```
def eval_poly(P, x):                                                    f903fa
    """evaluate a polynomial in x.
    :param P: an array representing the polynomial
    :returns: the value of P(x)
    :complexity: linear in the size of P.
    """
    # use Horner's rule
    retval = 0
    for pi in reversed(P):
        retval = retval * x + pi
    return retval
```

```
def add_poly(P, Q):                                                     456d28
    """ Add two polynomials represented by their coefficients.
    :param P, Q: two vectors representing polynomials
    :returns: a vector representing the addition
    :complexity: linear in the size of P and Q.
    """
    if len(P) < len(Q):
        P, Q = Q, P     # add the shorter to the longer vector
    R = P[::]           # make a copy
    for i, qi in enumerate(Q):
        R[i] += qi      # cumulate Q into R
    return R
```

```
def sub_poly(P, Q):                                                     5055a8
    """ Subtruct two polynomials represented by their coefficients.
    :param P, Q: two vectrs representing polynomials
    :returns: a vector representing the difference
    :complexity: linear in the size of P and Q.
    """
    return add_poly(P, [-qi for qi in Q])
```

```
def mul_poly(P, Q):                                                     f557bd
    """ Karatsuba's algorithm.
    Multiply two polynomials represented by their coefficients.
    i.e. P(x) = sum P[i] x**i.
    :param P, Q: two vectors representing polynomials
    :returns: a vector representing the product
    :complexity: $O(n^{\\log_2 3})=O(n^{1.585})$, where n is total degree of P and
```

```
Q.
    """
    if not P or not Q:  # case one of P, Q is the constant zero
        return []
    if len(P) == 1:
        return [qi * P[0] for qi in Q]
    elif len(Q) == 1:
        return [pi * Q[0] for pi in P]
    k = max(len(P), len(Q)) // 2
    xk = [0] * k
    a = P[:k]           # split: P = a + b * x**k
    b = P[k:]
    c = Q[:k]           # split: Q = c + d * x**k
    d = Q[k:]
    a_b = sub_poly(a, b)
    c_d = sub_poly(c, d)
    ac = mul_poly(a, c)
    bd = mul_poly(b, d)
    abcd = mul_poly(a_b, c_d)
    ad_bc = sub_poly(add_poly(ac, bd), abcd)    # = ad - bc
    # result is ac + [ac + bd - (a - b)(c - d)]*x**k + bd*x**(2k)
    return add_poly(ac, xk + add_poly(ad_bc, xk + bd))
```

## Knapsack

```
def knapsack(p, v, cmax):                                                    d9b896
    """Knapsack problem: select maximum value set of items if total size not
    more than capacity

    :param p: table with size of items
    :param v: table with value of items
    :param cmax: capacity of bag
    :requires: number of items non-zero
    :returns: value optimal solution, list of item indexes in solution
    :complexity: O(n * cmax), for n = number of items
    """
    n = len(p)
    opt = [[0] * (cmax + 1) for _ in range(n + 1)]
    sel = [[False] * (cmax + 1) for _ in range(n + 1)]
    #                           --- basic case
    for cap in range(p[0], cmax + 1):
        opt[0][cap] = v[0]
        sel[0][cap] = True
    #                           --- induction case
    for i in range(1, n):
        for cap in range(cmax + 1):
            if cap >= p[i] and opt[i-1][cap - p[i]] + v[i] > opt[i-1][cap]:
                opt[i][cap] = opt[i-1][cap - p[i]] + v[i]
                sel[i][cap] = True
            else:
                opt[i][cap] = opt[i-1][cap]
                sel[i][cap] = False
    #                           --- reading solution
    cap = cmax
    solution = []
    for i in range(n-1, -1, -1):
        if sel[i][cap]:
            solution.append(i)
            cap -= p[i]
    return (opt[n - 1][cmax], solution)
```

```
def knapsack_val(p, v, cmax):                                                eea858
    opt = [0] * (cmax + 1)
    for pi, vi in zip(p, v):
        for c in range(cmax, pi - 1, -1):
            opt[c] = max(opt[c], opt[c - pi] + vi)
    return opt[cmax]
```

## Knuth Morris Pratt




```
def maximum_border_length(w):                                                4b4d87
    """Maximum string borders by Knuth-Morris-Pratt

    :param w: string
    :returns: table f such that f[i] is the longest border length of w[:i + 1]
    :complexity: linear
    """
    n = len(w)
    f = [0] * n                 # init f[0] = 0
    k = 0                       # current longest border length
    for i in range(1, n):       # compute f[i]
        while w[k] != w[i] and k > 0:
            k = f[k - 1]        # mismatch: try the next border
        if w[k] == w[i]:        # last characters match
            k += 1              # we can increment the border length
        f[i] = k                # we found the maximal border of w[:i + 1]
    return f
```

```
def knuth_morris_pratt(s, t):                                                5187d9
    """Find a substring by Knuth-Morris-Pratt

    :param s: the haystack string
    :param t: the needle string
    :returns: index i such that s[i: i + len(t)] == t, or -1
    :complexity: O(len(s) + len(t))
    """
    sep = '\x00'                # special unused character
    assert sep not in t and sep not in s
    f = maximum_border_length(t + sep + s)
    n = len(t)
    for i, fi in enumerate(f):
        if fi == n:             # found a border of the length of t
            return i - 2 * n    # beginning of the border in s
    return -1
```

```python
def powerstring_by_border(u):                                    252254
    """Power string by Knuth-Morris-Pratt

    :param u: string
    :returns: largest k such that there is a string y with u = y^k
    :complexity: O(len(u))
    """
    f = maximum_border_length(u)
    n = len(u)
    if n % (n - f[-1]) == 0:       # does the alignment shift divide n ?
        return n // (n - f[-1])    # we found a power decomposition
    return 1
```

## Kruskal

```python
class UnionFind:                                                 b54756
    """Maintains a partition of {0, ..., n-1}
    """
    def __init__(self, n):
        self.up_bound = list(range(n))
        self.rank = [0] * n

    def find(self, x_index):
        """
        :returns: identifier of part containing x_index
        :complex_indexity: O(inverse_ackerman(n))
        """
        if self.up_bound[x_index] == x_index:
            return x_index
        self.up_bound[x_index] = self.find(self.up_bound[x_index])
        return self.up_bound[x_index]

    def union(self, x_index, y_index):
        """
        Merges part that contain x and part containing y
        :returns: False if x_index, y_index are already in same part
        :complexity: O(inverse_ackerman(n))
        """
        repr_x = self.find(x_index)
        repr_y = self.find(y_index)
        if repr_x == repr_y:       # already in the same component
            return False
        if self.rank[repr_x] == self.rank[repr_y]:
            self.rank[repr_x] += 1
            self.up_bound[repr_y] = repr_x
        elif self.rank[repr_x] > self.rank[repr_y]:
            self.up_bound[repr_y] = repr_x
        else:
            self.up_bound[repr_x] = repr_y
        return True
```

```python
def kruskal(graph, weight):                                      fd1634
    """Minimum spanning tree by Kruskal

    :param graph: undirected graph in listlist or listdict format
    :param weight: in matrix format or same listdict graph
    :returns: list of edges of the tree
    :complexity: ``O(|E|log|E|)``
    """
    u_f = UnionFind(len(graph))
    edges = []
    for u, _ in enumerate(graph):
        for v in graph[u]:
            edges.append((weight[u][v], u, v))
    edges.sort()
    min_span_tree = []
    for w_idx, u_idx, v_idx in edges:
        if u_f.union(u_idx, v_idx):
            min_span_tree.append((u_idx, v_idx))
    return min_span_tree
```

## Kuhn Munkres

```python
def kuhn_munkres(G, TOLERANCE=1e-6):                             6cc556
    """Maximum profit bipartite matching by Kuhn-Munkres

    :param G: weight matrix where G[u][v] is the weight of edge (u,v),
    :param TOLERANCE: a value with absolute value below tolerance
                      is considered as being zero.
                      If G consists of integer or fractional values
                      then TOLERANCE can be chosen 0.
    :requires: graph (U,V,E) is complete bi-partite graph with len(U) <= len(V)
               float('-inf') or float('inf') entries in G
               are allowed but not None.
    :returns: matching table from U to V, value of matching
    :complexity: :math:`O(|U|^2 |V|)`
    """
    nU = len(G)
    U = range(nU)
    nV = len(G[0])
    V = range(nV)
    assert nU <= nV
    mu = [None] * nU                # empty matching
    mv = [None] * nV
    lu = [max(row) for row in G]    # trivial labels
    lv = [0] * nV
    for root in U:                  # build an alternate tree
        au = [False] * nU           # au, av mark nodes...
        au[root] = True             # ... covered by the tree
        Av = [None] * nV            # Av[v] successor of v in the tree
        # for every vertex u, slack[u] := (val, v) such that
        # val is the smallest slack on the constraints (*)
        # with fixed u and v being the corresponding vertex
        slack = [(lu[root] + lv[v] - G[root][v], root) for v in V]
        while True:
            (delta, u), v = min((slack[v], v) for v in V if Av[v] is None)
            assert au[u]
            if delta > TOLERANCE:   # tree is full
                for u0 in U:        # improve labels
                    if au[u0]:
                        lu[u0] -= delta
```

```
                for v0 in V:
                    if Av[v0] is not None:
                        lv[v0] += delta
                    else:
                        (val, arg) = slack[v0]
                        slack[v0] = (val - delta, arg)
            assert abs(lu[u] + lv[v] - G[u][v]) <= TOLERANCE  # equality
            Av[v] = u                   # add (u, v) to A
            if mv[v] is None:
                break                   # alternating path found
            u1 = mv[v]
            assert not au[u1]
            au[u1] = True               # add (u1, v) to A
            for v1 in V:
                if Av[v1] is None:      # update margins
                    alt = (lu[u1] + lv[v1] - G[u1][v1], u1)
                    if slack[v1] > alt:
                        slack[v1] = alt
        while v is not None:            # ... alternating path found
            u = Av[v]                   # along path to root
            prec = mu[u]
            mv[v] = u                   # augment matching
            mu[u] = v
            v = prec
    return (mu, sum(lu) + sum(lv))
```

## Left Right Inversions




```
def _merge_sort(tab, tmp, rank, left, right, lo, hi):                    fd9a95
    if hi <= lo + 1:            # interval is empty or singleton
        return                  # nothing to do
    mid = lo + (hi - lo) // 2   # divide interval into [lo:mid], [mid:hi]
    _merge_sort(tab, tmp, rank, left, right, lo, mid)
    _merge_sort(tab, tmp, rank, left, right, mid, hi)
    i = lo                      # merge both lists
    j = mid
    k = lo
    while k < hi:
        if i < mid and (j == hi or tab[rank[i]] <= tab[rank[j]]):
            tmp[k] = rank[i]
            right[rank[i]] += j - mid
            i += 1
        else:
            tmp[k] = rank[j]
            left[rank[j]] += mid - i
            j += 1
        k += 1
    for k in range(lo, hi):     # copy sorted segment into original table
        rank[k] = tmp[k]
```

```
def left_right_inversions(tab):
    """ Compute left and right inversions of each element of a table.

    :param tab: list with comparable elements
    :returns: lists left and right. left[j] = the number of
        i<j such that tab[i] > tab[j].
        right[i] = the number of i<j such that tab[i] > tab[j].
    :complexity: `O(n log n)`
    """
    n = len(tab)
    left = [0] * n
    right = [0] * n
    tmp = [None] * n      # temporary table
    rank = list(range(n))
    _merge_sort(tab, tmp, rank, left, right, 0, n)
    return left, right
```

## Levenshtein




```
def levenshtein(x, y):                                                   457c45
    """Levenshtein edit distance

    :param x:
    :param y: strings
    :returns: distance
    :complexity: `O(|x|*|y|)`
    """
    n = len(x)
    m = len(y)
    # Create the table A
    #   Row 0 and column 0 are initialized as required
    #   The remaining entries will be overwritten during the computation
    A = [[i + j for j in range(m + 1)] for i in range(n + 1)]
    for i in range(n):
        for j in range(m):
            A[i + 1][j + 1] = min(A[i][j + 1] + 1,              # insert
                                  A[i + 1][j] + 1,              # delete
                                  A[i][j] + int(x[i] != y[j]))  # subst.
    return A[n][m]
```

## Longest Common Subsequence

```
def longest_common_subsequence(x, y):                                   bf3ce0
    """Longest common subsequence

    Dynamic programming

    :param x:
    :param y: x, y are lists or strings
```

```
    :returns: longest common subsequence in form of a string
    :complexity: `O(|x|*|y|)`
    """
    n = len(x)
    m = len(y)

    #                       -- compute optimal length
    A = [[0 for j in range(m + 1)] for i in range(n + 1)]
    for i in range(n):
        for j in range(m):
            if x[i] == y[j]:
                A[i + 1][j + 1] = A[i][j] + 1
            else:
                A[i + 1][j + 1] = max(A[i][j + 1],  A[i + 1][j])

    #                       -- extract solution in reverse order
    sol = []
    i, j = n, m
    while A[i][j] > 0:
        if A[i][j] == A[i - 1][j]:
            i -= 1
        elif A[i][j] == A[i][j - 1]:
            j -= 1
        else:
            i -= 1
            j -= 1
            sol.append(x[i])
    return ''.join(sol[::-1])  # reverse the list to obtain the solution
```

## Longest Increasing Subsequence

```python
from bisect import bisect_left                                  974435


def longest_increasing_subsequence(x):
    """Longest increasing subsequence

    :param x: sequence
    :returns: longest strictly increasing subsequence y
    :complexity: `O(|x|*log(|y|))`
    """
    n = len(x)
    p = [None] * n
    h = [None]
    b = [float('-inf')]  # - infinity
    for i in range(n):
        if x[i] > b[-1]:
            p[i] = h[-1]
            h.append(i)
            b.append(x[i])
        else:
            #   -- binary search: b[k - 1] < x[i] <= b[k]
            k = bisect_left(b, x[i])
            h[k] = i
            b[k] = x[i]
            p[i] = h[k - 1]
    # extract solution in reverse order
    q = h[-1]
    s = []
    while q is not None:
        s.append(x[q])
        q = p[q]
    return s[::-1]  # reverse the list to obtain the solution
```

## Lowest Common Ancestor

```python
class LCA:                                                      ee91d6
    def __init__(self, p):
        """ p[v] is predecessor of v in tree. Assuming p[v] < v, and p[0]=0=root
        """
        n = len(p)
        self.depth = [0] * n
        for u in range(1, n):
            self.depth[u] = self.depth[p[u]] + 1

        self.up = [p[:]]
        for _ in range(n.bit_length() - 1):
            a = self.up[-1]
            self.up.append([a[a[u]] for u in range(n)])

    def query(self, u, v):
        if self.depth[u] < self.depth[v]:
            u, v = v, u

        d = self.depth[u] - self.depth[v]
        for k, a in enumerate(self.up):
            if d & (1 << k):
                u = a[u]

        if u == v:
            return u

        for a in reversed(self.up):
            if a[u] != a[v]:
                u, v = a[u], a[v]

        return self.up[0][u]
```

## Matrix Chain Mult

```python
def matrix_mult_opt_order(M):                                   1bbe00
    """Matrix chain multiplication optimal order

    :param M: list of matrices
    :returns: matrices opt, arg, such that opt[i][j] is the optimal number of
              operations to compute M[i] * ... * M[j] when done in the order
              (M[i] * ... * M[k]) * (M[k + 1] * ... * M[j]) for k = arg[i][j]
    :complexity: :math:`O(n^3)`
    """
    n = len(M)
    r = [len(Mi) for Mi in M]
```

```
    c = [len(Mi[0]) for Mi in M]
    opt = [[0 for j in range(n)] for i in range(n)]
    arg = [[None for j in range(n)] for i in range(n)]
    for j_i in range(1, n):   # loop on i, j of increasing j - i = j_i
        for i in range(n - j_i):
            j = i + j_i
            opt[i][j] = float('inf')
            for k in range(i, j):
                alt = opt[i][k] + opt[k + 1][j] + r[i] * c[k] * c[j]
                if opt[i][j] > alt:
                    opt[i][j] = alt
                    arg[i][j] = k
    return opt, arg


def matrix_chain_mult(M):
    """Matrix chain multiplication

    :param M: list of matrices
    :returns: M[0] * ... * M[-1], computed in time optimal order
    :complexity: whatever is needed by the multiplications
    """
    opt, arg = matrix_mult_opt_order(M)
    return _apply_order(M, arg, 0, len(M)-1)


def _apply_order(M, arg, i, j):
    # --- multiply matrices from M[i] to M[j] included
    if i == j:
        return M[i]
    k = arg[i][j]        # --- follow placement of parentheses
    A = _apply_order(M, arg, i, k)
    B = _apply_order(M, arg, k + 1, j)
    row_A = range(len(A))
    row_B = range(len(B))
    col_B = range(len(B[0]))
    return [[sum(A[a][b] * B[b][c] for b in row_B)
             for c in col_B] for a in row_A]
```

## Merge Ordered Lists

```
def merge(x, y):                                                    61a3ee
    """Merge two ordered lists

    :param x:
    :param y: x, y are nondecreasing ordered lists
    :returns: union of x and y in order
    :complexity: linear
    """
    z = []
    i = 0
    j = 0
    while i < len(x) or j < len(y):
        if j == len(y) or i < len(x) and x[i] <= y[j]:  # priority on x
            z.append(x[i])
            i += 1
        else:
            z.append(y[j])                              # now switch to y
            j += 1
    return z
```

## Min Mean Cycle

```
def min_mean_cycle(graph, weight, start=0):                         deac32
    """Minimum mean cycle by Karp

    :param graph: directed graph in listlist or listdict format
    :param weight: in matrix format or same listdict graph
    :param int start: vertex that should be contained in cycle
    :returns: cycle as vertex list, average arc weights
              or None if there is no cycle from start
    :complexity:  `O(|V|*|E|)`
    """
    INF = float('inf')
    n = len(graph)                  # compute distances
    dist = [[INF] * n]
    prec = [[None] * n]
    dist[0][start] = 0
    for ell in range(1, n + 1):
        dist.append([INF] * n)
        prec.append([None] * n)
        for node in range(n):
            for neighbor in graph[node]:
                alt = dist[ell - 1][node] + weight[node][neighbor]
                if alt < dist[ell][neighbor]:
                    dist[ell][neighbor] = alt
                    prec[ell][neighbor] = node
    #                               -- find the optimal value
    valmin = INF
    argmin = None
    for node in range(n):
        valmax = -INF
        argmax = None
        for k in range(n):
            alt = (dist[n][node] - dist[k][node]) / float(n - k)
            # do not divide by float(n-k) => cycle of minimal total weight
            if alt >= valmax:     # with >= we get simple cycles
                valmax = alt
                argmax = k
        if argmax is not None and valmax < valmin:
            valmin = valmax
            argmin = (node, argmax)
    #                               -- extract cycle
    if valmin == INF:             # -- there is no cycle
        return None
    C = []
    node, k = argmin
    for l in range(n, k, -1):
        C.append(node)
        node = prec[l][node]
    return C[::-1], valmin
```

## Next Permutation

409758
```
def next_permutation(tab):
    """find the next permutation of tab in the lexicographical order

    :param tab: table with n elements from an ordered set
    :modifies: table to next permutation
    :returns: False if permutation is already lexicographical maximal
    :complexity: O(n)
    """
    n = len(tab)
    pivot = None                            # find pivot
    for i in range(n - 1):
        if tab[i] < tab[i + 1]:
            pivot = i
    if pivot is None:                       # tab is already the last perm.
        return False
    for i in range(pivot + 1, n):           # find the element to swap
        if tab[i] > tab[pivot]:
            swap = i
    tab[swap], tab[pivot] = tab[pivot], tab[swap]
    i = pivot + 1
    j = n - 1                               # invert suffix
    while i < j:
        tab[i], tab[j] = tab[j], tab[i]
        i += 1
        j -= 1
    return True
```

## Pareto

79b651
```
def pareto2d(points):
    """ Compute the Pareto set of a given set of points in 2 dimensions

    :param points: list of tuples with the coordinates of the points. Can be
floating point coordinates.
    :modifies: points will be sorted
    :returns: a list of non-dominated points
    :complexity: $O(n\\log n)$
    """
    pareto = []
    points.sort()
    for p in points:
        x, y = p
        if pareto == [] or y < pareto[-1][1] or p == pareto[-1]:
            pareto.append(p)
    return pareto
```

4db31b
```
def pareto3d(points):
    """ Compute the Pareto set of a given set of points in 3 dimensions

    :param points: list of tuples with the coordinates of the points. Can be
floating point coordinates.
    :modifies: points will be sorted
    :returns: a list of non-dominated points
    :complexity: $O(n\\log n)$
    """
    # compute the ranks, it is ok to have multiple y-values in the list
    y_values = [y for x,y,z in points]
    y_values.sort()
    rank = {}
    for i, yi in enumerate(y_values):
        rank[yi] = i
    n = len(points)
    points.sort()    # sort by rank in first competition
    pareto = []
    R = FenwickMin(n)
    for p in points:
        x, y, z = p
        i = rank[y]
        if pareto == [] or R.prefixMin(i) > z or p == pareto[-1]:
            pareto.append(p)
        R.update(i, z)
    return pareto
```

## Polygon

88b95d
```
def area(p):
    """Area of a polygone

    :param p: list of the points taken in any orientation,
              p[0] can differ from p[-1]
    :returns: area
    :complexity: linear
    """
    A = 0
    for i, _ in enumerate(p):
        A += p[i - 1][0] * p[i][1] - p[i][0] * p[i - 1][1]
    return A / 2.
```

b2fd3e
```
def is_simple(polygon):
    """Test if a rectilinear polygon is is_simple

    :param polygon: list of points as (x,y) pairs along the closed polygon
    :returns: True if the segements do not intersect
    :complexity: O(n log n) for n=len(polygon)
    """
    n = len(polygon)
    order = list(range(n))
    order.sort(key=lambda i: polygon[i])         # lexicographic order
    rank_to_y = list(set(p[1] for p in polygon))
    rank_to_y.sort()
    y_to_rank = {y: rank for rank, y in enumerate(rank_to_y)}
    S = RangeMinQuery([0] * len(rank_to_y))      # sweep structure
    last_y = None
    for i in order:
        x, y = polygon[i]
        rank = y_to_rank[y]
        #                               -- type of point
        right_x = max(polygon[i - 1][0], polygon[(i + 1) % n][0])
```

```
        left = x < right_x
        below_y = min(polygon[i - 1][1], polygon[(i + 1) % n][1])
        high = y > below_y
        if left:                        # y does not need to be in S yet
            if S[rank]:
                return False            # two horizontal segments intersect
            S[rank] = -1                # add y to S
        else:
            S[rank] = 0                 # remove y from S
        if high:
            lo = y_to_rank[below_y]     # check S between [lo + 1, rank - 1]
            if (below_y != last_y or last_y == y or
                    rank - lo >= 2 and S.range_min(lo + 1, rank)):
                return False            # horiz. & vert. segments intersect
        last_y = y                      # remember for next iteration
    return True
```

## Primes

```
def eratosthene(n):                                                    49160b
    """Prime numbers by sieve of Eratosthene

    :param n: positive integer
    :assumes: n > 2
    :returns: list of prime numbers <n
    :complexity: O(n loglog n)
    """
    P = [True] * n
    answ = [2]
    for i in range(3, n, 2):
        if P[i]:
            answ.append(i)
            for j in range(i * i, n, 2 * i):
                P[j] = False
    return answ
```

```
def gries_misra(n):                                                    e14321
    """Prime numbers by the sieve of Gries-Misra
    Computes both the list of all prime numbers less than n,
    and a table mapping every integer 2 ≤ x < n to its smallest prime factor

    :param n: positive integer
    :returns: list of prime numbers, and list of prime factors
    :complexity: O(n)
    """
    primes = []
    factor = [0] * n
    for x in range(2, n):
        if not factor[x]:       # no factor found
            factor[x] = x       # meaning x is prime
            primes.append(x)
        for p in primes:        # loop over primes found so far
            if p > factor[x] or p * x >= n:
                break
            factor[p * x] = p  # p is the smallest factor of p * x
    return primes, factor
```

## Rabin Karp

```
PRIME = 72057594037927931     # < 2^{56}                                 b3a252
DOMAIN = 128


def roll_hash(old_val, out_digit, in_digit, last_pos):
    """roll_hash """
    val = (old_val - out_digit * last_pos + DOMAIN * PRIME) % PRIME
    val = (val * DOMAIN) % PRIME
    return (val + in_digit) % PRIME
```

```
def matches(s, t, i, j, k):                                            b3d2a9
    """
    Checks whether s[i:i + k] is equal to t[j:j + k].
    We used a loop to ease the implementation in other languages.
    """
    # tests if s[i:i + k] equals t[j:j + k]
    for d in range(k):
        if s[i + d] != t[j + d]:
            return False
    return True
```

```
def rabin_karp_matching(s, t):                                         e03349
    """Find a substring by Rabin-Karp

    :param s: the haystack string
    :param t: the needle string

    :returns: index i such that s[i: i + len(t)] == t, or -1
    :complexity: O(len(s) + len(t)) in expected time,
                 and O(len(s) * len(t)) in worst case
    """
    hash_s = 0
    hash_t = 0
    len_s = len(s)
    len_t = len(t)
    last_pos = pow(DOMAIN, len_t - 1) % PRIME
    if len_s < len_t:               # substring too long
        return -1
    for i in range(len_t):          # preprocessing
        hash_s = (DOMAIN * hash_s + ord(s[i])) % PRIME
        hash_t = (DOMAIN * hash_t + ord(t[i])) % PRIME
    for i in range(len_s - len_t + 1):
        if hash_s == hash_t:                # hashes match
            # check character by character
            if matches(s, t, i, 0, len_t):
                return i
        if i < len_s - len_t:
            # shift window and calculate new hash on s
            hash_s = roll_hash(hash_s, ord(s[i]), ord(s[i + len_t]),
```

```
                                last_pos)
    return -1                                   # no match
```

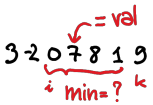
```
def rabin_karp_factor(s, t, k):                                    1500fa
    """Find a common factor by Rabin-Karp

    :param string s: haystack
    :param string t: needle
    :param int k: factor length
    :returns: (i, j) such that s[i:i + k] == t[j:j + k] or None.
              In case of tie, lexicographical minimum (i, j) is returned
    :complexity: O(len(s) + len(t)) in expected time,
                 and O(len(s) + len(t) * k) in worst case
    """
    last_pos = pow(DOMAIN, k - 1) % PRIME
    pos = {}
    assert k > 0
    if len(s) < k or len(t) < k:
        return None
    hash_t = 0

    # First calculate hash values of factors of t
    for j in range(k):
        hash_t = (DOMAIN * hash_t + ord(t[j])) % PRIME
    for j in range(len(t) - k + 1):
        # store the start position with the hash value
        if hash_t in pos:
            pos[hash_t].append(j)
        else:
            pos[hash_t] = [j]
        if j < len(t) - k:
            hash_t = roll_hash(hash_t, ord(t[j]), ord(t[j + k]), last_pos)

    hash_s = 0
    # Now check for matching factors in s
    for i in range(k):          # preprocessing
        hash_s = (DOMAIN * hash_s + ord(s[i])) % PRIME
    for i in range(len(s) - k + 1):
        if hash_s in pos:       # is this signature in s?
            for j in pos[hash_s]:
                if matches(s, t, i, j, k):
                    return (i, j)
        if i < len(s) - k:
            hash_s = roll_hash(hash_s, ord(s[i]), ord(s[i + k]), last_pos)
    return None
```

## Range Minimum Query




```
class RangeMinQuery:                                               9426b0
    """Range minimum query

    maintains a table t, can read/write items t[i],
    and query range_min(i,k) = min{ t[i], t[i + 1], ..., t[k - 1]}
    :complexity: all operations in O(log n), for n = len(t)
    """
    def __init__(self, t, INF=float('inf')):
        self.INF = INF
        self.N = 1
        while self.N < len(t):                      # find size N
            self.N *= 2
        self.s = [self.INF] * (2 * self.N)
        for i in range(len(t)):                     # store values of t
            self.s[self.N + i] = t[i]               # in the leaf nodes
        for p in range(self.N - 1, 0, -1):          # fill inner nodes
            self.s[p] = min(self.s[2 * p], self.s[2 * p + 1])

    def __getitem__(self, i):
        return self.s[self.N + i]

    def __setitem__(self, i, v):
        """ sets t[i] to v.
            :complexity: O(log len(t))
        """
        p = self.N + i
        self.s[p] = v
        p //= 2                                     # climb up the tree
        while p > 0:                                # update node
            self.s[p] = min(self.s[2 * p], self.s[2 * p + 1])
            p //= 2

    def range_min(self, i, k):
        """:returns:  min{ t[i], t[i + 1], ..., t[k - 1]}
        :complexity: O(log len(t))
        """
        return self._range_min(1, 0, self.N, i, k)

    def _range_min(self, p, start, span, i, k):
        """returns the minimum in t in the indexes [i, k) intersected
           with [start, start + span).
           p is the node associated to the later interval.
        """
        if start + span <= i or k <= start:         # disjoint intervals
            return self.INF
        if i <= start and start + span <= k:        # contained intervals
            return self.s[p]
        left = self._range_min(2 * p, start, span // 2,
                               i, k)
        right = self._range_min(2 * p + 1, start + span // 2, span // 2,
                                i, k)
        return min(left, right)
```

```
class LazySegmentTree:                                             bf34ce
    """maintains a tree to allow quick updates and queries on a table.
```

```
    This is more general than a Fenwick tree or a tree for MinRangeQuery. Here
    queries and updates act on index ranges. Updates can be set a range to a
    value or add a value to a range. Queries can be max, min and sum over an
    index range. All operations run in time O(log n) for a the table size n.
    The given ranges are in the form [i,j] where i is included and j excluded.
    In the recursive calls, node is the index of a node in the tree, and left,
    right its range. Values can be any numerical values allowing max, min, and
    sum, such as integers, floating point numbers or fractions (from the class
    Fraction). Updates over an empty range is valid and does nothing. Queries
    over an empty range is valid and returns the neutral value -inf, +inf or
    0.

    If the node is cleared, then maxval, minval, sumval represent for each
    node the query responses over the corresponding index ranges.  If the node
    is not clean, it means that lazyset and/or lazyadd contain suspendet
    update instructions for that node. Clearing a node means propagating these
    values to the descents in the subtrees, and updating maxval,minval and
    sumval for that node.
    """
    def __init__(self, tab):
        """stores an integer table tab.
        will be padded to get a table with a size of a power of 2.

        :param array tab: of positive length
        """
        self.N = 1
        while self.N < len(tab):
            self.N *= 2
        self.maxval = [float('-inf')] * 2 * self.N  # init with neutral values
        self.minval = [float('+inf')] * 2 * self.N
        self.sumval = [0] * 2 * self.N
        self.lazyset = [None] * 2 * self.N
        self.lazyadd = [0] * 2 * self.N
        for i, tabi in enumerate(tab):            # initialize with given table
            j = self.N + i
            self.maxval[j] = self.minval[j] = self.sumval[j] = tabi
        for node in range(self.N - 1, 0, -1):
            self._maintain(node)                  # maintain invariant

    def _maintain(self, node):
        """maintains the invariant for the given node
        :promize: the lazy values are None/0 for this node
        """
        # requires node and its direct descends to be clean
        ll = 2 * node
        r = 2 * node + 1
        assert self.lazyset[node] is None
        assert self.lazyadd[node] == 0
        assert self.lazyset[ll] is None
        assert self.lazyadd[ll] == 0
        assert self.lazyset[r] is None
        assert self.lazyadd[r] == 0
        self.maxval[node] = max(self.maxval[ll], self.maxval[r])
        self.minval[node] = min(self.minval[ll], self.minval[r])
        self.sumval[node] = self.sumval[ll] + self.sumval[r]

    def _clear(self, node, left, right):
        """propagates the lazy updates for this node to the subtrees.
        as a result the maxval, minval, sumval values for the node
        are up to date.
        """
        if self.lazyset[node] is not None:  # first do the pending set
            val = self.lazyset[node]
            self.minval[node] = val
            self.maxval[node] = val
            self.sumval[node] = val * (right - left)
            self.lazyset[node] = None
            if left < right - 1:            # not a leaf
                self.lazyset[2 * node] = val  # propagate to direct childs
                self.lazyadd[2 * node] = 0
                self.lazyset[2 * node + 1] = val
                self.lazyadd[2 * node + 1] = 0
        if self.lazyadd[node] != 0:         # then do the pending add
            val = self.lazyadd[node]
            self.minval[node] += val
            self.maxval[node] += val
            self.sumval[node] += val * (right - left)
            self.lazyadd[node] = 0
            if left < right - 1:            # not at a leaf
                self.lazyadd[2 * node] += val  # propagate to direct childs
                self.lazyadd[2 * node + 1] += val

    def add(self, i, j, val):
        self._add(i, j, val, 1, 0, self.N)

    def set(self, i, j, val):
        self._set(i, j, val, 1, 0, self.N)

    def max(self, i, j):
        return self._max(i, j, 1, 0, self.N)

    def min(self, i, j):
        return self._min(i, j, 1, 0, self.N)

    def sum(self, i, j):
        return self._sum(i, j, 1, 0, self.N)

    def _add(self, i, j, val, node, left, right):
        self._clear(node, left, right)
        if j <= left or right <= i:
            return   # disjoint intervals, nothing to do
        if i <= left and right <= j:
            self.lazyadd[node] += val
            self._clear(node, left, right)
        else:
            mid = (right + left) // 2
            self._add(i, j, val, 2 * node, left, mid)
            self._add(i, j, val, 2 * node + 1, mid, right)
            self._maintain(node)

    def _set(self, i, j, val, node, left, right):
        self._clear(node, left, right)
        if j <= left or right <= i:
            return   # disjoint intervals, nothing to do
```

```
        if i <= left and right <= j:
            self.lazyset[node] = val
            self.lazyadd[node] = 0
            self._clear(node, left, right)
        else:
            mid = (right + left) // 2
            self._set(i, j, val, 2 * node, left, mid)
            self._set(i, j, val, 2 * node + 1, mid, right)
            self._maintain(node)

    def _max(self, i, j, node, left, right):
        if j <= left or right <= i:
            return float('-inf')   # neutral value for max
        self._clear(node, left, right)
        if i <= left and right <= j:
            return self.maxval[node]
        else:
            mid = (right + left) // 2
            a = self._max(i, j, 2 * node, left, mid)
            b = self._max(i, j, 2 * node + 1, mid, right)
            return max(a, b)

    def _min(self, i, j, node, left, right):
        if j <= left or right <= i:
            return float('+inf')   # neutral value for min
        self._clear(node, left, right)
        if i <= left and right <= j:
            return self.minval[node]
        else:
            mid = (right + left) // 2
            a = self._min(i, j, 2 * node, left, mid)
            b = self._min(i, j, 2 * node + 1, mid, right)
            return min(a, b)

    def _sum(self, i, j, node, left, right):
        if j <= left or right <= i:
            return 0               # neutral value for sum
        self._clear(node, left, right)
        if i <= left and right <= j:
            return self.sumval[node]
        else:
            mid = (right + left) // 2
            a = self._sum(i, j, 2 * node, left, mid)
            b = self._sum(i, j, 2 * node + 1, mid, right)
            return a + b
```

## Rectangles From Grid

```
def rectangles_from_grid(P, black=1):                                    1c204b
    """Largest area rectangle in a binary matrix

    :param P: matrix
    :param black: search for rectangles filled with value black
    :returns: area, left, top, right, bottom of optimal rectangle
             consisting of all (i,j) with
             left <= j < right and top <= i <= bottom
    :complexity: linear
    """
    rows = len(P)
    cols = len(P[0])
    t = [0] * cols
    best = None
    for i in range(rows):
        for j in range(cols):
            if P[i][j] == black:
                t[j] += 1
            else:
                t[j] = 0
        (area, left, height, right) = rectangles_from_histogram(t)
        alt = (area, left, i, right, i-height)
        if best is None or alt > best:
            best = alt
    return best
```

## Rectangles From Histogram

```
def rectangles_from_histogram(H):                                        16e106
    """Largest Rectangular Area in a Histogram

    :param H: histogram table
    :returns: area, left, height, right, rect. is [0, height] * [left, right)
    :complexity: linear
    """
    best = (float('-inf'), 0, 0, 0)
    S = []
    H2 = H + [float('-inf')]  # extra element to empty the queue
    for right, _ in enumerate(H2):
        x = H2[right]
        left = right
        while len(S) > 0 and S[-1][1] >= x:
            left, height = S.pop()
            # first element is area of candidate
            rect = (height * (right - left), left, height, right)
            if rect > best:
                best = rect
        S.append((left, x))
    return best
```

## Strongly Connected Components

```
def kosaraju_dfs(graph, nodes, order, sccp):                             94e274
    times_seen = [-1] * len(graph)
    for start in nodes:
        if times_seen[start] == -1:                    # initiate DFS
            to_visit = [start]
            times_seen[start] = 0
            sccp.append([start])
            while to_visit:
                node = to_visit[-1]
                children = graph[node]
                if times_seen[node] == len(children):   # end of process
```

```
                    to_visit.pop()
                    order.append(node)
                else:
                    child = children[times_seen[node]]
                    times_seen[node] += 1
                    if times_seen[child] == -1:         # new node
                        times_seen[child] = 0
                        to_visit.append(child)
                        sccp[-1].append(child)


def reverse(graph):
    """replace all arcs (u, v) by arcs (v, u) in a graph"""
    V = range(len(graph))
    rev_graph = [[] for node in V]
    for node in V:
        for neighbor in graph[node]:
            rev_graph[neighbor].append(node)
    return rev_graph


def kosaraju(graph):
    """Strongly connected components by Kosaraju

    :param graph: directed graph in listlist format, cannot be listdict
    :returns: list of lists for each component
    :complexity: linear
    """
    V = range(len(graph))
    order = []
    sccp = []
    kosaraju_dfs(graph, V, order, [])
    kosaraju_dfs(reverse(graph), order[::-1], [], sccp)
    return sccp[::-1]  # follow inverse topological order
```

## Subsetsum

```
def subset_sum(x, R):                                                    f89efc
    """Subsetsum

    :param x: table of non negative values
    :param R: target value
    :returns bool: True if a subset of x sums to R
    :complexity: O(n*R)
    """
    b = [False] * (R + 1)
    b[0] = True
    for xi in x:
        for s in range(R, xi - 1, -1):
            b[s] |= b[s - xi]
    return b[R]
```

```
def coin_change(x, R):                                                   8c101f
    """Coin change

    :param x: table of non negative values
    :param R: target value
    :returns bool: True if there is a non negative linear combination
        of x that has value R
    :complexity: O(n*R)
    """
    b = [False] * (R + 1)
    b[0] = True
    for xi in x:
        for s in range(xi, R + 1):
            b[s] |= b[s - xi]
    return b[R]
```

## Suffix Array

```
abanan 5
anaban 3
ananab 1
banana 0
nabana 4
nanaba 2
```

```
def sort_class(s):                                                       222a55
    """ sorts s and returns additional information

    :param s: string or list
    :returns p, c: p[j]=i if s[i] has rank j in sorted(s) and c[i] is rank of s[i]
in sorted(set(s))
    :complexity: O(n log n) or better if sort makes use of specific values in s
    """
    S_index = [(x, i) for i, x in enumerate(s)]
    p = [i for x, i in sorted(S_index)]
    x2c = {x : i for i, x in enumerate(sorted(set(s)))}
    c = [x2c[x] for x in s]
    return p, c
```

```
def sort_cyclic_shifts(s):                                               a0bd65
    """ given a string s, sort lexicographically all cyclic shifts of s.

    The i-th cyclic shift of s is s[i:] + s[i:]
    :param s: string or list
    :returns L: such that L[j]=i if the i-th cyclic shift of s has rank j
    :complexity: O(n * log(n)^2)
    """
    p, c = sort_class(s)
    n = len(s)
    K = 1
    while K <= n:
        L = [(c[i], c[(i + K) % n]) for i in range(n)]
        p, c = sort_class(L)
        K <<= 1
    return p
```

```
def suffix_array(s):                                                     d233fe
    """ given a string s, sort lexicographically suffixes of s
```

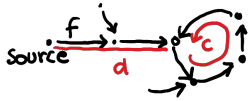
```
    :param s: string
    :returns: R with R[i] is j such that s[j:] has rank i
    :complexity: O(n log^2 n)
    """
    special = chr(0)
    assert special < min(s)
    L = sort_cyclic_shifts(s + special)
    return L[1:]
```

## Topological Order

```
def topological_order(graph):                                          c7423f
    """Topological sorting by maintaining indegree

    :param graph: directed graph in listlist format, cannot be listdict
    :returns: list of vertices in order
    :complexity: `O(|V|+|E|)`
    """
    V = range(len(graph))
    indeg = [0 for _ in V]
    for node in V:            # compute indegree
        for neighbor in graph[node]:
            indeg[neighbor] += 1
    Q = [node for node in V if indeg[node] == 0]
    order = []
    while Q:
        node = Q.pop()        # node without incoming arrows
        order.append(node)
        for neighbor in graph[node]:
            indeg[neighbor] -= 1
            if indeg[neighbor] == 0:
                Q.append(neighbor)
    return order
```

## Tortoise Hare




```
def tortoise_hare(f, source=0):                                        af3c54
    """ Detect cycle for function f, starting from source

    :param f: function from finite domain to itself
    :param source: element in this domain
    :warning: if the function does not reach a cycle from source
              then this function loops forever
    :returns: c, d such that after d iterations
              of f a cycle is reached, which has period c
    :complexity: `O(d+c)`
    """
    t = f(source)       # tortoise
    h = f(f(source))    # hare
    # move to some position in cycle
    while t != h:
        t = f(t)
        h = f(f(h))
    # detect begining of cycle
    t = source
    d = 0
    while t != h:
        t = f(t)
        h = f(h)
        d += 1
    # detect period of cycle
    c = 1
    t = f(t)
    while t != h:
        t = f(t)
        c += 1
    return c, d
```

## Trie

```
from string import ascii_letters    # in Python 2 one would import letters   ae9843


class TrieNode:
    def __init__(self):                             # each node will have
        self.is_word = False                        # 52 children -
        self.s = {c: None for c in ascii_letters}   # most will remain empty


def add(T, w, i=0):  # Add a word to the trie
    """
    :param T: trie
    :param string w: word to be added to T
    :returns: new trie consisting of w added into T
    :complexity: O(len(w))
    """
    if T is None:
        T = TrieNode()
    if i == len(w):
        T.is_word = True
    else:
        T.s[w[i]] = add(T.s[w[i]], w, i + 1)
    return T


def Trie(S):  # Build the trie for the words in the dictionary S
    """
    :param S: set of words
    :returns: trie containing all words from S
    :complexity: linear in total word sizes from S
    """
    T = None
    for w in S:
        T = add(T, w)
    return T
```

```
def spell_check(T, w):  # Spell check a word against the trie
    """Spellchecker

    :param T: trie encoding the dictionary
    :param w: given word
    :returns: a closest word from the dictionary
    :complexity: linear if distance was constant
    """
    assert T is not None
    dist = 0
    while True:
        u = search(T, dist, w)
        if u is not None:  # Match at distance dist
            return u
        dist += 1          # No match - try increasing the distance


def search(T, dist, w, i=0):
    """Searches for w[i:] in trie T with distance at most dist
    """
    if i == len(w):
        if T is not None and T.is_word and dist == 0:
            return ""
        else:
            return None
    if T is None:
        return None
    f = search(T.s[w[i]], dist, w, i + 1)       # matching
    if f is not None:
        return w[i] + f
    if dist == 0:
        return None
    for c in ascii_letters:
        f = search(T.s[c], dist - 1, w, i)      # insertion
        if f is not None:
            return c + f
        f = search(T.s[c], dist - 1, w, i + 1)  # substitution
        if f is not None:
            return c + f
    return search(T, dist - 1, w, i + 1)        # deletion
```

## Two Sat

```
def _vertex(lit):  # integer encoding of a litteral                    e18d40
    if lit > 0:
        return 2 * (lit - 1)
    return 2 * (-lit - 1) + 1


def two_sat(formula):
    """Solving a 2-SAT boolean formula

    :param formula: list of clauses, a clause is pair of literals
                    over X1,...,Xn for some n.
                    a literal is an integer, for example -1 = not X1, 3 = X3
    :returns: table with boolean assignment satisfying the formula or None
    :complexity: linear
    """
    # num_variables is the number of variables
    num_variables = max(abs(clause[p])
                        for p in (0, 1) for clause in formula)
    graph = [[] for node in range(2 * num_variables)]
    for x_idx, y_idx in formula:                            # x_idx or y_idx
        graph[_vertex(-x_idx)].append(_vertex(y_idx))     # -x_idx => y_idx
        graph[_vertex(-y_idx)].append(_vertex(x_idx))     # -y_idx => x_idx
    sccp = tarjan(graph)
    comp_id = [None] * (2 * num_variables)  # each node's component ID
    assignment = [None] * (2 * num_variables)
    for component in sccp:
        rep = min(component)             # representative of the component
        for vtx in component:
            comp_id[vtx] = rep
            if assignment[vtx] is None:
                assignment[vtx] = True
                assignment[vtx ^ 1] = False    # complementary literal
    for i in range(num_variables):
        if comp_id[2 * i] == comp_id[2 * i + 1]:
            return None                        # insatisfiable formula
    return assignment[::2]
```

## Windows K Distinct

```
def windows_k_distinct(x, k):                                          7c6fa6
    """Find all largest windows containing exactly k distinct elements

    :param x: list or string
    :param k: positive integer
    :yields: largest intervals [i, j) with len(set(x[i:j])) == k
    :complexity: `O(|x|)`
    """
    dist, i, j = 0, 0, 0                # dist = |{x[i], ..., x[j-1]}|
    occ = {xi: 0 for xi in x}           # number of occurrences in x[i:j]
    while j < len(x):
        while dist == k:                # move start of interval
            occ[x[i]] -= 1              # update counters
            if occ[x[i]] == 0:
                dist -= 1
            i += 1
        while j < len(x) and (dist < k or occ[x[j]]):
            if occ[x[j]] == 0:          # update counters
                dist += 1
            occ[x[j]] += 1
            j += 1                      # move end of interval
        if dist == k:
            yield (i, j)                # one interval found
```